

SIT213

Simulation d'une chaîne de transmission numérique

Etape 1

Chaîne de transmission logique parfaite

Auteurs : Ziani Amine
Sissoko Mohamed
Nanda Laurent

Groupe : B — FIP2A

Date : Septembre 2026

Contexte

L'objectif du projet SIT213 est de simuler de bout en bout une chaîne de transmission numérique. On part d'un message binaire généré à la source, on le fait passer à travers un canal, et on le récupère à destination. À chaque étape du projet, on mesure la qualité de la transmission via le **Taux d'Erreur Binaire (TEB)**.

Dans cette première étape, on travaille avec un canal parfait : pas de bruit, pas de perte de bits. Le but est de mettre en place les bases de l'architecture avant d'y ajouter de la complexité dans les étapes suivantes.

Architecture

La chaîne de transmission de l'étape 1 est la suivante :

```
Source    -->    TransmetteurParfait    -->    DestinationFinale
```

Le professeur nous a fourni les classes abstraites (`Source`, `Transmetteur`, `Destination`), les interfaces et les outils de visualisation (sondes). Notre travail a consisté à implémenter les classes concrètes manquantes et à compléter le `Simulateur`.

Implémentation

SourceFixe

Génère un message binaire à partir d'une chaîne de caractères imposée (ex : "0110101"). Chaque caractère '1' est converti en `true`, chaque '0' en `false`. Elle est utilisée quand l'option `-mess` reçoit une suite d'au moins 7 caractères binaires.

SourceAleatoire

Génère un message binaire aléatoire de longueur `nbBits`. On peut lui passer une semence (`seed`) pour reproduire exactement le même tirage d'une exécution à l'autre, ce qui est utile pour comparer des résultats ou déboguer.

TransmetteurParfait

Reçoit les bits en entrée et les retransmet tels quels à toutes les destinations connectées, sans aucune modification. Il implémente les deux méthodes abstraites héritées de `Transmetteur` :

- `recevoir()` stocke l'information reçue et appelle immédiatement `emettre()`.
- `emettre()` propage l'information à toutes les destinations connectées en sortie.

DestinationFinale

Mémoire l'information reçue en bout de chaîne dans l'attribut `informationRecue`. C'est cette information qu'on compare ensuite avec ce qui a été émis pour calculer le TEB.

Simulateur

Trois parties étaient à compléter dans le squelette fourni :

Le constructeur : on y crée la source (fixe ou aléatoire selon les arguments reçus), le transmetteur parfait et la destination finale, puis on les connecte dans l'ordre. Si l'option `-s` est active, on branche également des sondes graphiques (`SondeLogique`) pour visualiser le signal en sortie de la source et du transmetteur.

`execute()` : appelle simplement `source.emettre()`, ce qui déclenche le parcours de toute la chaîne en cascade.

`calculTauxErreurBinaire()` : compare bit par bit le message émis et le message reçu, et retourne le rapport entre le nombre de bits incorrects et le nombre total de bits.

Résultats

Autotests

On a lancé le script `runTests` fourni avec le projet :

```
=== Autotests simulateur ===
[OK] message aleatoire par default (longueur 100)
[OK] message binaire impose de longueur 7
[OK] message aleatoire de longueur imposee (20)
[OK] TEB nul avec -seed
[OK] argument -mess invalide
[OK] option non reconnue

=== Bilan : 6 OK, 0 KO ===
```

Les 6 tests passent. Les cas valides donnent tous un TEB de 0.0, ce qui est attendu pour un transmetteur parfait. Les cas d'erreur lèvent correctement une `ArgumentsException`.

Exemples de simulation

```
java Simulateur
=> TEB : 0.0

java Simulateur -mess 0110101
=> TEB : 0.0

java Simulateur -mess 50 -seed 42
=> TEB : 0.0

java Simulateur -mess 0110101 -s
=> TEB : 0.0 (+ affichage des signaux emis et recus)
```

Conclusion

L'étape 1 est fonctionnelle. La chaîne de transmission logique parfaite est en place et tous les autotests passent avec un TEB de 0.0. L'architecture mise en place est conçue pour être étendue facilement : les étapes suivantes consisteront à remplacer le **TransmetteurParfait** par des transmetteurs analogiques avec différentes formes d'onde (NRZ, NRZT, RZ), puis à introduire du bruit gaussien, des trajets multiples et un codeur de canal.